

Web Services: An introduction

Thanks to Eleni Stroulia, Ken
Birman, Yossi Amir, Itzik Kasovitch

Agenda

- Introduction
 - Motivation and Background
 - Service-Oriented Architecture (SOA)
 - Distributed objects
- Technical details
 - Stack of standards: SOAP, WSDL, WSFL/XLANG/BPEL, UDDI
 - An example of deploying and using a web service using Axis
- Semantic-Web
 - Vision
 - Specifications
- Adoption status
- Conclusions

Agenda

- Motivation
- History
- Web service model
- Web service components
- A walkthrough examples

Motivation

- The ability to program the Web.
- Example: Consider an Excel spreadsheet that summarizes your whole financial picture : stocks, bank accounts, loans, etc. If some of this information is available through XML Web services, Excel can update it and present the update information to the user.

The Now and the Future

- The Web was designed for humans
 - It is specified in terms of few and “forgiving” standards (http + html)
 - It is supported on various (all) computing platforms
- It has been extremely successful
 - It is ubiquitous
 - It has enabled information sharing and B2C e-commerce.
- However, businesses need
 - to integrate disparate distributed resources;
 - to take advantage of supplier offers and to deliver customer requests;
 - to customize for special situations; and
 - to interact without having to exchange detailed information about their interfaces.
- Distributed-object solutions have had limited success due (to a great extent) to their proprietary nature.
- Enterprise application integration, automated business-process adaptation and B2B interactions are still difficult.

Web Service definition

A simple definition:

“a Web Service is an application component accessible over open protocols”.

Web Service Definition

- A software component that can be
 - **Described** using a service-description language, which
 - is in formal XML notation,
 - covers all the details necessary to interact with the service (message formats for operations, transport protocols and location), and
 - hides the implementation details of the service;
 - **Published** to a registry of services;
 - **Discovered** through a standard mechanism;
 - **Invoked** through a declared API, usually through a network; and
 - **Composed** with other services
 - enabling loosely coupled, component-oriented, cross-technology application implementations.

Programming Interfaces and APIs

- API = Application Programming Interface
 - ***Don't confuse this with a user Interface***
- Defines a set of functions that may be called by application programs
 - Like a library
 - But application developer may not have access to the code implementing the functions
 - E.g., being an eBay developer means you can use the API, not that you can see how it is implemented
 - And the functions may even be executed on a different computer!
 - E.g., eBay developers are using functions located on eBay servers, not their own computers

Example: eBay's API

Alternative to interacting with the web site

- Functions provided for
 - Listing items
 - Tracking a particular user's auctions
 - Leaving feedback
 - *Everything eBay users can do*
- Allows application developer to provide custom interface to users, with extra features
 - Built on top of eBay functionality

An eBay API ([click here](#))

LeaveFeedback: Call `LeaveFeedback` to post positive, negative, or neutral feedback information about a user after the conclusion of an auction.

- What input?
 - User: identified by a unique `userid`
 - Feedback: positive, negative, or neutral
- What output?
 - Success or error
- Write it
 - `LeaveFeedback(userID,feedback) → {Success | ErrorCode}`

What is a Google API?



- The Google Web APIs service is a beta web program that allows developers to easily find and manipulate information on the web.
- Google Web APIs are for developers and researchers interested in using Google as a resource in their applications.
- The Google Web APIs service allows software developers to query more than 3 billion web documents directly from their own computer programs.
- Google uses the SOAP and WSDL standards to act as an interface between the user's program and Google API.
- Programming environments such as Java, Perl, Visual Studio .NET are compatible with Google API.

Definitions from [http:// www.google.com/apis/](http://www.google.com/apis/)

What can you do with the API

- Developers can issue search requests to Google's index of more than 3 billion web pages.
 - and receive results as
 - structured data,
 - Estimated number of results, URL's, Snippets, Query Time etc.
 - access information in the Google cache,
 - and check the spelling of words.
- Google API

Evolution

- Web services evolved from previous technologies that served the same purpose such as RPC, ORPC (DCOM, CORBA and JAVA RMI).
- Web Services were intended to solve three main problems:
 1. Interoperability
 2. Firewall traversal
 3. Complexity

Interoperability

- Earlier distributed systems suffered from interoperability issues because each vendor implemented its own on-wire format for distributed object messaging.
- Development of DCOM apps strictly bound to Windows Operating system.
- Development of RMI bound to Java programming language.

<http://dret.net/glossary/>

Web and XML Glossary

1 2 3 4 5 6 7 A B C D E F G H I J K L M N O P Q
R S T U V W X Y Z

Welcome to the Web and XML Glossary

Welcome to [dret](#)'s Web and XML glossary. The acronym index (shown below) provides a concise list of all acronyms, while the individual letter indices (always accessible via the letter bar) list the full names as well. If you are interested, you can look at some [Statistics about the Glossary](#).

Distributed Object models

- DCOM is a Protocol that enables software components to communicate directly over a network in a reliable, secure, and efficient manner. Previously called OLE, DCOM is designed for use across multiple network transports, including Internet Protocols such as HTTP.
- RMI is an RPC mechanism enabling Java programmers to create distributed applications, in which the methods of remote Java objects can be invoked from another JVM, possibly on a different host.
- CORBA describes the architecture of a middleware platform that supports the implementation of applications in distributed and heterogeneous environments. The CORBA standard is issued by OMG. In contrast to other middleware platforms such as Microsoft's DCOM, CORBA is a specification that does not prescribe any specific technology.

Firewall traversal

- Collaboration across corporations was an issue because distributed systems such as CORBA and DCOM used non-standard ports.
- Web Services use HTTP as a transport protocol and most of the firewalls allow access through port 80 (HTTP), leading to easier and dynamic collaboration.

Complexity

- Web Services is a developer-friendly service system.
- Most of the above-mentioned technologies such as RMI, COM, and CORBA involve a whole learning curve.
- New technologies and languages have to be learnt to implement these services.

Web Service definition revisited

- A more precise definition:
 - an application component that:
 - Communicates via open protocols (HTTP, SMTP, etc.)
 - Processes XML messages framed using SOAP
 - Describes its messages using XML Schema
 - Provides an endpoint description using WSDL
 - Can be discovered using UDDI

Web Services Components

- **XML** – eXtensible Markup Language – A uniform data representation and exchange mechanism.
- **SOAP** – Simple Object Access Protocol – A standard way for communication.
- **UDDI** – Universal Description, Discovery and Integration specification – A mechanism to register and locate WS based application.
- **WSDL** – Web Services Description Language – A standard meta language to described the services offered.

Example – A simple Web Service

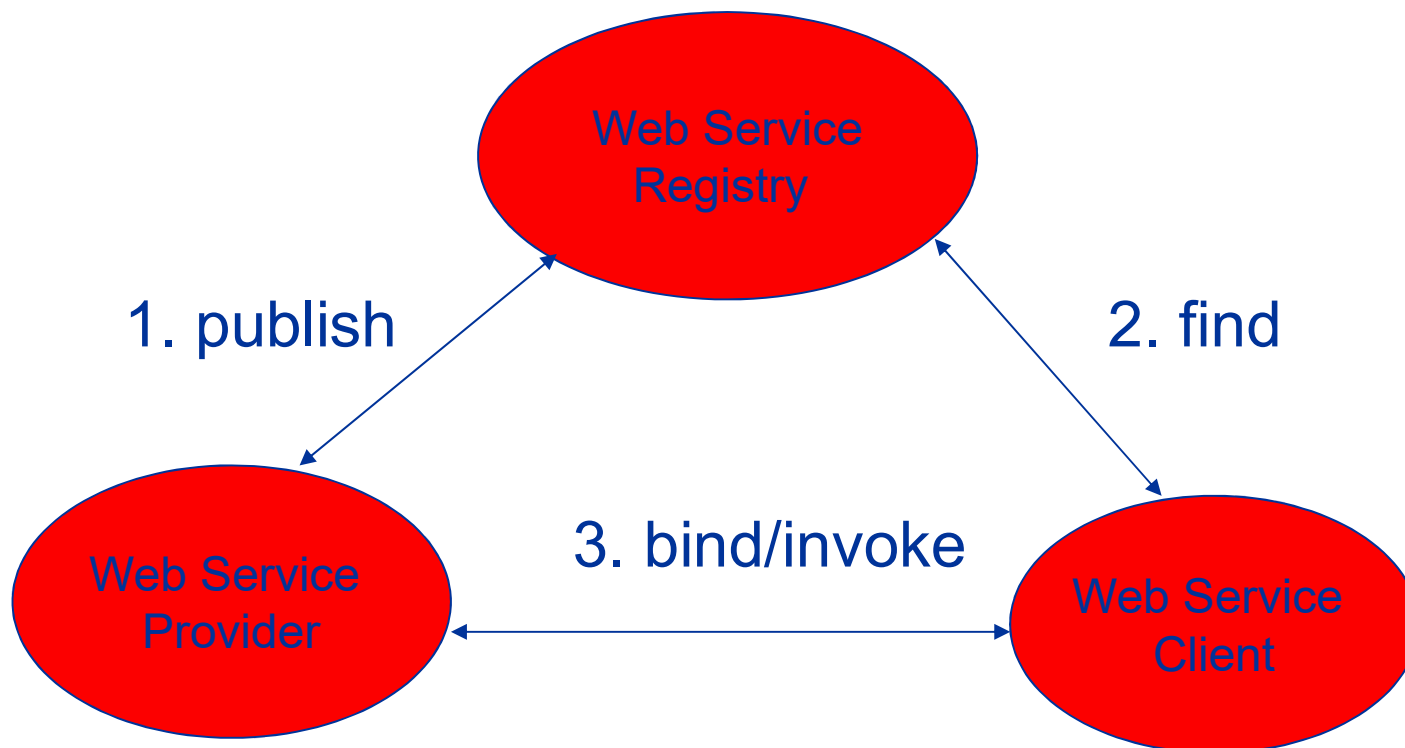
- A buyer (which might be a simple client) is ordering goods from a seller service.
- The buyer finds the seller service by searching the UDDI directory.
- The seller service is a Web Service whose interface is defined using Web Services Description Language (WSDL).
- The buyer is invoking the order method on the seller service using Simple Object Access Protocol (SOAP) and the WSDL definition for the seller service.
- The buyer knows what to expect in the SOAP reply message because this is defined in the WSDL definition for the seller service.

The Web Service Model

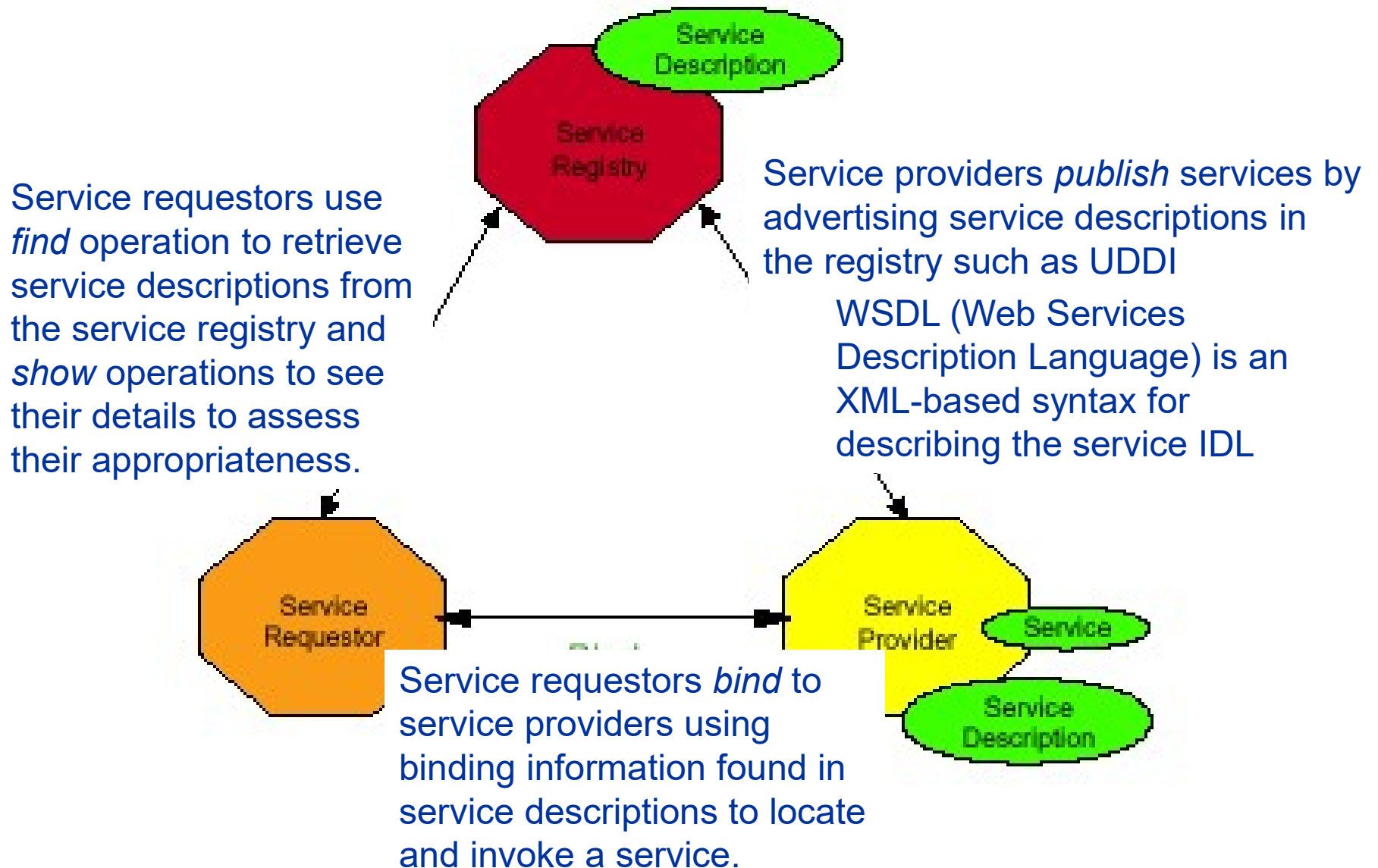
- The Web Services architecture is based upon the interactions between three roles:
 - Service provider
 - Service registry
 - Service requestor
- The interactions involve the:
 - Publish operations
 - Find operation
 - Bind operations.

The Web Service Model (cont)

The Web Services model follows the *publish, find, and bind* paradigm.



Service-Oriented Architectures



XML

- XML stands for **EX**tensible **M**arkup **L**anguage.
- XML is a **markup language** much like HTML.
- XML was designed to **describe data**.
- XML tags are not predefined. You must **define your own tags**.
- The perfect choice for enabling cross-platform data communication in Web Services.

XML vs HTML

An HTML example:

```
<html>
<body>
  <h2>John Doe</h2>
  <p>2 Backroads Lane<br>
    New York<br>
    045935435<br>
    john.doe@gmail.com<br>
  </p>
</body>
</html>
```

XML vs HTML

- This will be displayed as:

John Doe

2 Backroads Lane

New York

045935435

John.doe@gmail.com

- HTML specifies how the document is to be displayed, and not what information is contained in the document.
- Hard for machine to extract the embedded information. Relatively easy for human.

XML vs HTML

- Now look at the following:

```
<?xml version=1.0?>
<contact>
  <name>John Doe</name>
  <address>2 Backroads Lane</address>
  <country>New York</country>
  <phone>045935435</phone>
  <email>john.doe@gmail.com</email>
</contact>
```

- In this case:
 - The information contained is being marked, but not for displaying.
 - Readable by both human and machines.

SOAP

- SOAP originally stood for "Simple Object Access Protocol" .
- Web Services expose useful functionality to Web users through a standard Web protocol called SOAP.
- Soap is an XML vocabulary standard to enable programs on separate computers to interact across any network. SOAP is a simple markup language for describing messages between applications.
- Soap uses mainly HTTP as a transport protocol. That is, HTTP message contains a SOAP message as its payload section.

SOAP Characteristics

- SOAP has three major characteristics:
 - Extensibility – security and WS-routing are among the extensions under development.
 - Neutrality - SOAP can be used over any transport protocol such as HTTP, SMTP or even TCP.
 - Independent - SOAP allows for any programming model .

SOAP Building Blocks

A SOAP message is an ordinary XML document containing the following elements:

- A required Envelope element that identifies the XML document as a SOAP message.
- An optional Header element that contains header information.
- A required Body element that contains call and response information.
- An optional Fault element that provides information about errors that occurred while processing the message.

SOAP Request

POST /InStock HTTP/1.1

Host: www.stock.org

Content-Type: application/soap+xml; charset=utf-8 Content-Length: 150

```
<?xml version="1.0"?>
```

```
<soap:Envelope
```

```
  xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
```

```
  soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
```

```
    <soap:Body xmlns:m="http://www.stock.org/stock">
```

```
      <m:GetStockPrice>
```

```
        <m:StockName>IBM</m:StockName>
```

```
      </m:GetStockPrice>
```

```
    </soap:Body>
```

```
</soap:Envelope>
```


SOAP Response

HTTP/1.1 200 OK

Content-Type: application/soap; charset=utf-8

Content-Length: 126

```
<?xml version="1.0"?>
```

```
<soap:Envelope xmlns:soap="http://www.w3.org/2001/12/soap-  
envelope" soap:encodingStyle="http://www.w3.org/2001/12/soap-  
encoding">
```

```
  <soap:Body xmlns:m="http://www.stock.org/stock">
```

```
    <m:GetStockPriceResponse>
```

```
      <m:Price>34.5</m:Price>
```

```
    </m:GetStockPriceResponse>
```

```
  </soap:Body>
```

```
</soap:Envelope>
```

SOAP Security

- SOAP uses HTTP as a transport protocol and hence can use HTTP security mainly HTTP over SSL.
- But, since SOAP can run over a number of application protocols (such as SMTP) security had to be considered.
- The [WS-Security specification](#) defines a complete encryption system.

WSDL

- WSDL stands for Web Services Description Language.
- WSDL is an XML vocabulary for describing Web services. It allows developers to describe Web Services and their capabilities, in a standard manner.
- WSDL specifies what a request message must contain and what the response message will look like in unambiguous notation. In other words, it is a contract between the XML Web service and the client who wishes to use this service.
- In addition to describing message contents, WSDL defines where the service is available and what communications protocol is used to talk to the service.

The WSDL Document Structure

- A WSDL document is just a simple XML document.
- It defines a web service using these major elements:
 - **port type** - The operations performed by the web service.
 - **message** - The messages used by the web service.
 - **types** - The data types used by the web service.
 - **binding** - The communication protocols used by the web service.

WSDL Document

```
<message name="GetStockPriceRequest">
  <part name="stock" type="xs:string"/>
</message>
<message name="GetStockPriceResponse">
  <part name="value" type="xs:string"/>
</message>

<portType name="StocksRates">
  <operation name="GetStockPrice">
    <input message="GetStockPriceRequest"/>
    <output message="GetStockPriceResponse"/>
  </operation>
</portType>
```

UDDI

- UDDI stands for Universal Description, Discovery and Integration.
- UDDI is a directory for storing information about web services , like yellow pages.
- UDDI is a directory of web service interfaces described by WSDL.

Resources

- <http://msdn.microsoft.com/webservices/understanding/webservicebasics/default.aspx>
- <http://www.w3schools.com/>
- <http://uddi.microsoft.com/Default.aspx>
- <http://www.developer.com/services/article.php/2195981>
- Many more on the web...

Web Services vs. Distributed Objects

So what's new here?

Isn't this what CORBA does?

<http://weblogs.cs.cornell.edu/AllThingsDistributed/archives/000120.html>

[skip](#)

Web services - Distributed objects: Similarities

- Both have some sort of description language
 - A bridge mechanism is necessary between client and server, defining
 - What to call: operations, signatures, return types, exceptions
 - How to make an invocation, in each “bridged” environment
 - Compilers generate client proxy and server skeleton
 - Run-time middleware mediates the client-server interaction
- Both have well-defined network interactions
- Both have a similar mechanism for registering and discovering available components

Web services - Distributed objects: Differences

- The client-server interaction patterns are different: the client object obtains a reference to the server object through a factory object. The web-service client sends messages to the service based on information available in the service WSDL.
- Distributed objects enable stateful computing; the server lifecycle depends on the client-server interaction. Web services - at their basic incarnation - are designed for stateless computing.
- A client object can hold a reference to a server and access the server state through the server's lifetime. There is no web-service reference mechanism.
- Distributed objects were designed mainly for within an intranet, and were conceived as decentralization technology. Web services are intended as a technology in support of integration on the web.

Implementing Web-Services with Apache Axis: An Introduction

<http://ws.apache.org/axis/>

[skip](#)

- The Java API for XML-based remote procedure calls (JAX-RPC) simplifies the process of building Web services that incorporate XML-based RPC.
- It defines mappings between Java types and XML types that attempt to hide the details of XML and provide a familiar method-call paradigm.
- We will use JAX-RPC to implement and call SOAP-based Web services described by the Web Services Description Language (WSDL) using Apache's open source tools
 - Apache Tomcat for deployment
 - Apache Axis for SOAP implementation

Web-service application architecture (behavior)

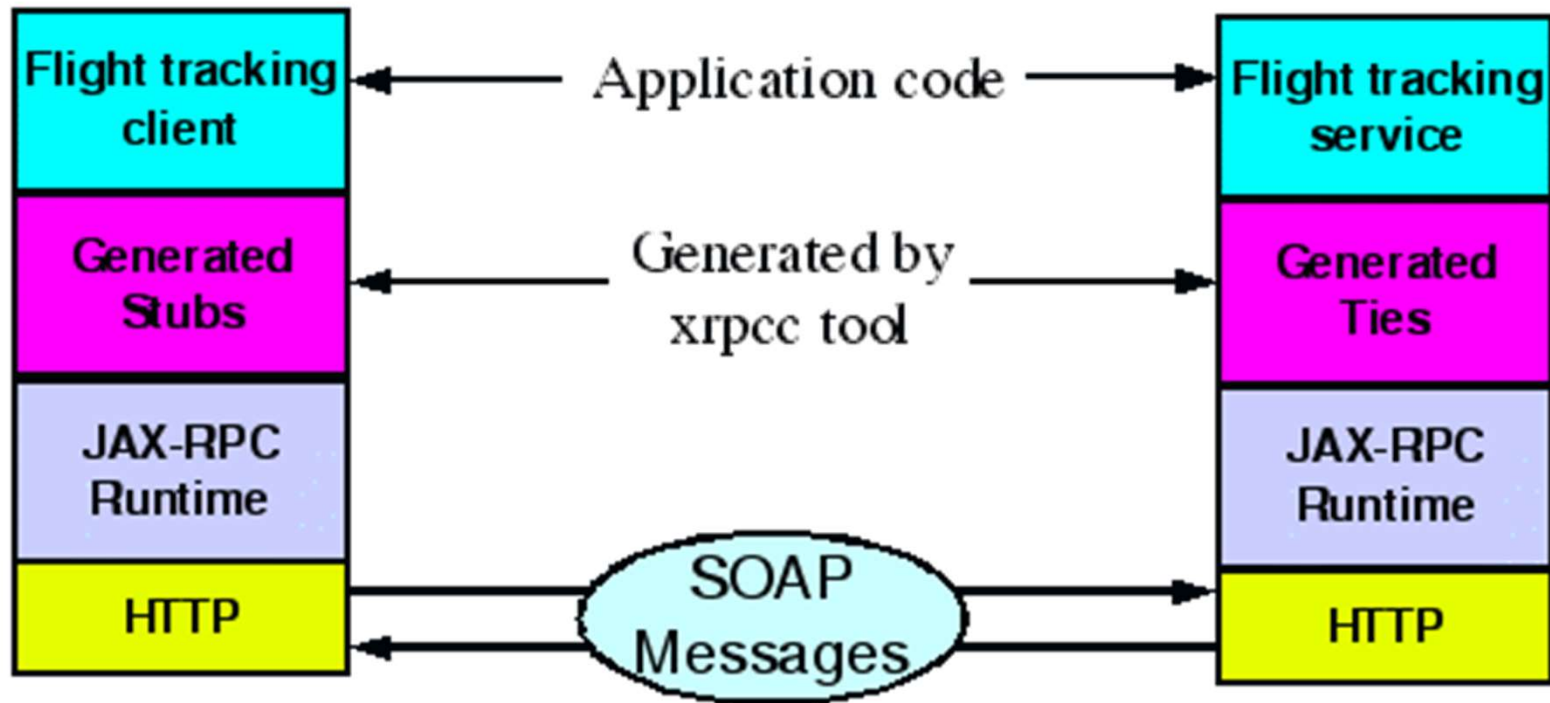
QuickTime™ and a TIFF (Uncompressed) decompressor are needed to see this picture.

<http://www.developer.com/java/web/article.php/2237251>

How does it work?

- On the client side (Application 1)
 - The Java support packages (e.g., *org.apache.axis.client*)
 - generate the appropriate SOAP request - according to the server that the client is accessing -
 - send it to the Axis engine as an HTTP request,
 - receive the resulting SOAP response,
 - process it, and
 - return the de-serialized return java object to the original calling client method.
- On the server side (Application 2)
 - The Axis engine provides all of the support to
 - process the SOAP request,
 - call the indicated method in the deployed service class,
 - receive its return java object,
 - package it in a SOAP response, and
 - return it via HTTP to the client.

Web-service application architecture (structure)



<http://www.sosnoski.com/presents/java-xml/axis/index.html>

A simple example

<http://www.cs.unc.edu/Courses/comp190/index.html>

[skip](#)

The java server class

```
public class MessageObject extends java.lang.Object
{
    protected java.lang.String message;
    public MessageObject() {
        message = "Hello, World, from jbs
MessageObject"; }

    public void setMessage(java.lang.String message)
    {
        this.message = message; }

    public java.lang.String getMessage() {
        return this.message; }
}
```

- A simple java class
 - Contains one single protected field, message
 - Has three methods
 - Constructor: MessageObject
 - Setter: setMessage
 - Getter: getMessage

The SOAP Client

```
public class SOAP_Call {  
    public static String callService() {  
        String msg = null;  
        try {  
            String endpoint =  
                "http://jbs.cs.unc.edu:8888/axis/MessageObject.jws";  
            Service service = new Service();  
            Call call = (Call) service.createCall();  
            call.setTargetEndpointAddress(new java.net.URL(endpoint) );  
            call.setOperationName(new QName("http://soapinterop.org/",  
                "getMessage") );  
            msg = (String) call.invoke( new Object[]{} ); }  
        catch (Exception e) { System.err.println(e.toString()); }  
        return msg; }  
    public SOAP_Call() { super(); }  
}
```

- This client relies on the `org.apache.axis.client` package
- A Service object is created
 - It is used to create a Call object
- Several parameters are set on the Call object:
 - `call.setTargetEndpointAddress` sets the URL for the endpoint (the Axis engine plus deployed MessageObject class)
 - `call.setOperationName` sets the name of the method Axis is to call, structured as a namespace-qualified name.
 - here, `soapinterop.org`
 - if `getMessage` required parameters, they would also be set in a similar manner
- The service is invoked

The SOAP request

```
POST /axis/MessageObject.jws HTTP/1.0
Host: localhost
Content-Type: text/xml; charset=utf-8
SOAPAction: ""
Content-Length: 448
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
  soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:soapenv=" http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd=" http://www.w3.org/2001/XMLSchema"
  xmlns:xsi=" http://www.w3.org/2001/XMLSchema-instance"
  xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/">
  <soapenv:Body>
    <ns1:getMessage xmlns:ns1="http://soapinterop.org/" />
  </soapenv:Body>
</soapenv:Envelope>
```

- The SOAP request is carried in the body of an HTTP request
- The only unusual aspect of the SOAP request is all of the namespaces referenced.
 - Those in the envelope are conventional and most are expected by the SOAP engine (e.g., encodingStyle, soapenv, xsd, xsi, and SOAP-ENC).
 - The soapinterop.org namespace appears in the body and is required because of the QName reference in the client; that is, it qualifies getMessage.

The SOAP response

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
Date: Tue, 17 Sep 2002 15:35:36 GMT
Server: Apache Tomcat/4.0.4 (HTTP/1.1 Connector)
Connection: close
Set-Cookie: JSESSIONID=115C2B8879FCC1E2603BE3;Path=/axis
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Body>
    <ns1:getMessageResponse
      soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
      xmlns:ns1="http://soapinterop.org/">
      <getMessageReturn xsi:type="xsd:string">Hello, World, from jbs
      MessageObject</getMessageReturn>
    </ns1:getMessageResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

- Similarly, the SOAP response is carried in the body of an HTTP response
- The envelope namespaces are conventional.
- In the body, the tag for the response is constructed from the name of the method with Response appended.
- Within it, the actual data value is bounded by another constructed tag: the name of the method with Return appended, indicating that the value is that returned by the method called.
- Note also that its type is indicated as xsd:string.

Resources

- Graham et al. 3-4
- Developing JAX-RPC–Based Web Services Using Axis and SOAP
 - <http://www.developer.com/java/web/article.php/2237251>
- A tutorial by Sosnoski
 - <http://www.sosnoski.com/presents/java-xml/axis/index.html>
- Creating Web Services with AXIS - Apache's latest SOAP implementation bootstraps Web services
 - <http://www.linux-mag.com/cgi-bin/printer.pl?issue=2002-08&article=axis>

Semantic Web

http://www.sciam.com/print_version.cfm?articleID=00048144-10D2-1C70-84A9809EC588EF21

<http://www.w3.org/2000/Talks/1206-xml2k-tbl/Overview-2.html>

Levels of Interoperability

Frank Manola: Interoperability Issues in
Large-Scale Distributed Object Systems.
ACM Computing Surveys 27 (2): 268-270 (1995)

Level	Agreement
■ physical • sharing data	■ marshaling
■ object model • sharing objects	■ ORB architectures • OMG IDL • Microsoft OLE, ...
■ cooperation of sets of classes	■ Frameworks
■ semantic • sharing the meaning of data and operations	■ metadata • data ontology • algorithms • expected side-effects

The vision

“The Semantic Web will bring structure to the meaningful content of Web pages, creating an environment where **software agents** roaming from page to page can readily carry out **sophisticated tasks for users**. Such an agent coming to the clinic's Web page will know **not just that the page has keywords** such as "treatment, medicine, physical, therapy" (as might be encoded today) but also that Dr. Hartman *works* at this *clinic* on *Mondays, Wednesdays* and *Fridays* and that the script takes a *date range* in *yyyy-mm-dd* format and returns *appointment times*. And it will "know" all this without needing artificial intelligence on the scale of 2001's Hal or Star Wars's C-3PO. Instead these semantics were encoded into the Web page when the clinic's office manager (who never took Comp Sci 101) massaged it into shape **using off-the-shelf software for writing Semantic Web pages** along with **resources listed on the Physical Therapy Association's site**.”

<http://www.scientificamerican.com/article.cfm?articleID=00048144-10D2-1C70-84A9809EC588EF21&catID=2>

The KR prerequisites

- Logic is necessary to support
 - the representation of complex, structured collections of information
 - conducting automated reasoning through inference rules
- Ontologies
- intelligent-agent collaboration
 - Semantic services

Ontologies

- An ontology is a theory about what types of things exist.
- For the semantic web an ontology is a document that formally defines the relations among terms.
 - The most typical kind of ontology for the Web has
 - a taxonomy
 - An inheritance forest among terms
 - a set of inference rules
 - Additional relation rules "If a city code is associated with a state code, and an address uses that city code, then that address has the associated state code."
 - (possibly) a set of equivalence relations
 - Equivalent terms can exploit each other's inheritance and other relations

<http://www-ksl.stanford.edu/kst/what-is-an-ontology.html>

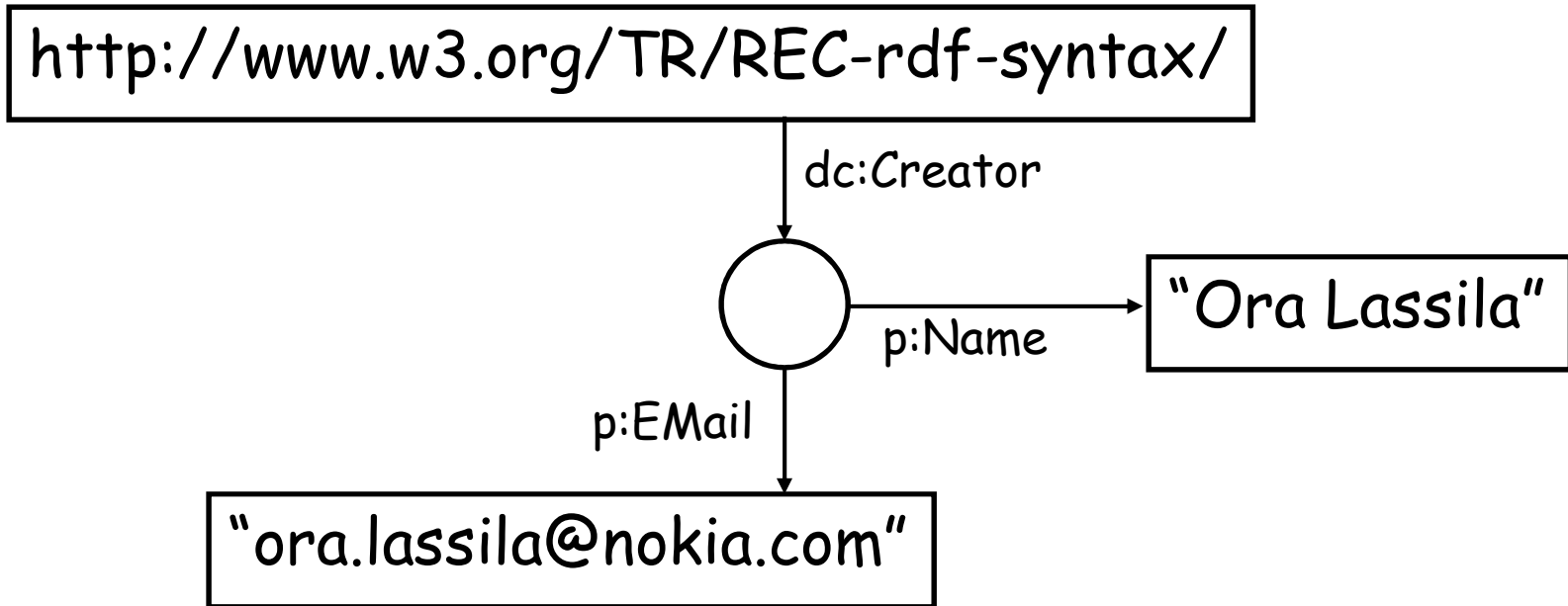
Why is XML not enough for ontology specification?

- XML makes no commitment on:
 - Domain-specific vocabulary
 - The vocabulary of the tags chosen for a schema is up to the designer and (possibly) the agreement of the community
 - Ontological modeling primitives
 - Schemas offer limited syntax for expressing semantic constraints, such as
 - Equivalence
 - Inheritance

RDF

- RDF encodes meaning in sets of triples, written using XML tags
- In RDF, a document makes assertions that
 - particular things (people, Web pages or whatever)
 - have properties ("is a sister of," "is the author of")
 - with certain values (another person, another Web page)
- Subject, verb and object are each identified by a Universal Resource Identifier (URI)

RDF example



- A graph node (corresponding to a resource) also can be the value of a property
- Corresponding triples
 - { `"http://www.w3.org/TR/PR-rdf-syntax/"`, `dc:Creator`, `x` }
 - { `x`, `p:Name`, `"Ora Lassila"` }
 - { `x`, `p:EMail`, `"ora.lassila@nokia.com"` }

Beyond RDF: OIL + DAML = OWL

- OIL extends RDF Schema to a fully-fledged knowledge representation language.
 - logical expressions
 - data-typing
 - cardinality
 - quantifiers
- DAML = US sister of OIL
- OWL, the merge of DAML+OIL, W3C candidate recommendation February 2004

<http://www.w3.org/TR/owl-features/>

OWL-S Services Ontology

- The class Service is at the top of the OWL-S ontology.
- The ontology of services provides two essential types of knowledge about a service
 - Profile
 - Model
- The ServiceProfile provides information about
 - (1) Functionality
 - (2) Constraints (security, locality, affordability)
- The ServiceModel enables an agent to
 - (1) perform a more in-depth analysis of whether the service meets its needs;
 - (2) compose multiple service descriptions to perform a specific task;
 - (3) coordinate the activities of different agents; and
 - (4) monitor the execution of the service.

<http://www.daml.org/services/owl-s/1.0/>

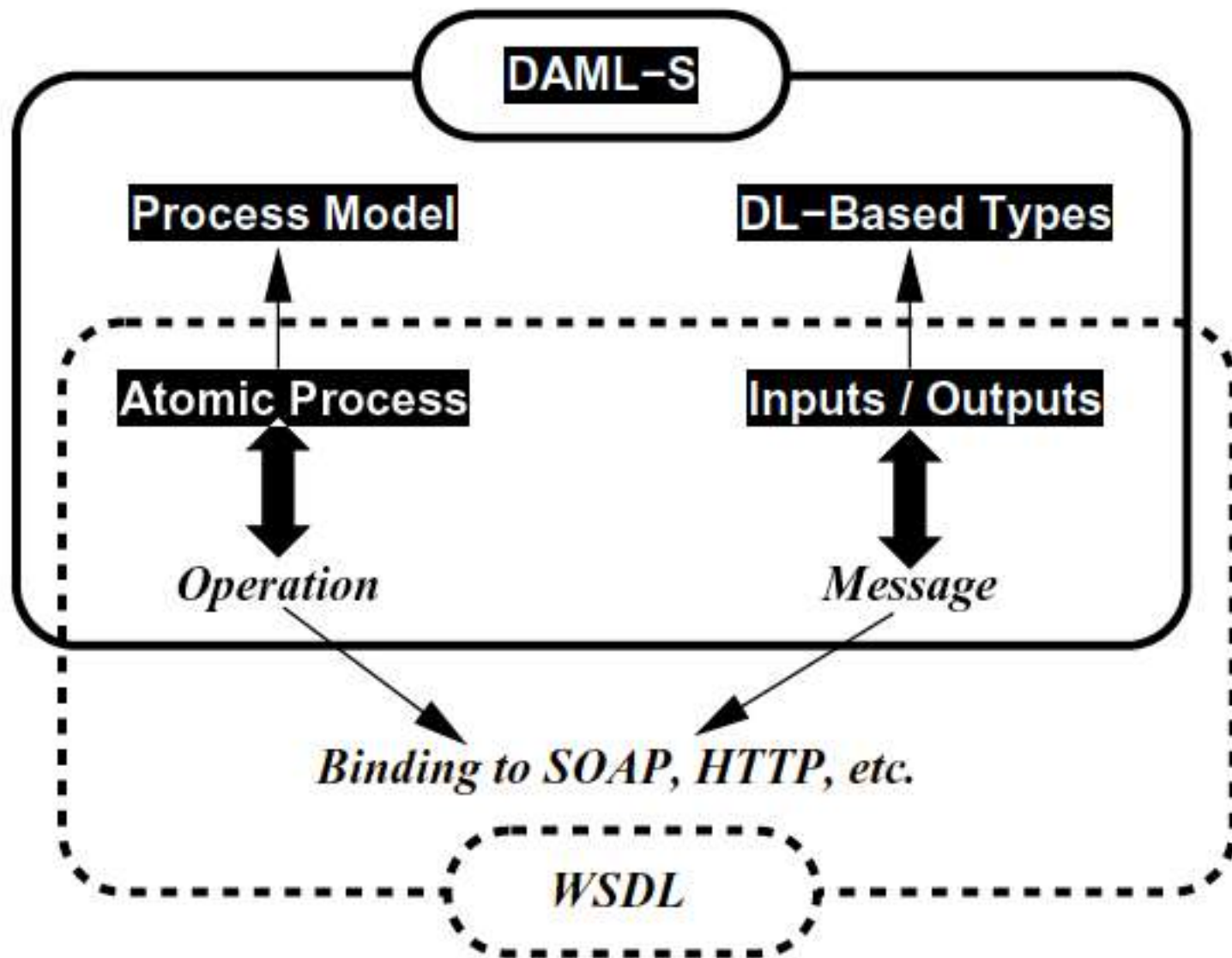


Fig. 1. Mapping between DAML-S and WSDL

Web Services Adoption: A status Report

Resources

- Gartner Surveys Show Web Services Are Entering the Mainstream
 - http://www4.gartner.com/DisplayDocument?doc_cd=114570
 - April 2003, amalgamated results of several surveys
- Adoption of Web Services and Technology Choices, TechMetrix Research
 - <http://www.techmetrix.com/products/products.php?type=rep>
 - July 2001, February 2003, Questionnaire to TechMetrix subscribers (610 collected)
- Web Services usage survey, CBDI report
 - http://www.cbdiforum.com/bronze/webserv_usage/webserv_usage.php3
 - March 2003, surveys CBDI subscribers practical experience
- Web Services Roadmap - Guiding the Transition to Web Services and SOA, CBDI report
 - <http://www-3.ibm.com/software/solutions/webservices/>

Gartner Dataquest survey - 1

- 92% of system integration projects — and those projected through February 2004 — involve Web services
- 86% of enterprises use XML
- But, other Web services standards are much less used:
 - SOAP (Simple Object Access Protocol) (31%);
 - WSDL (Web Service Definition Language) (3%);
 - UDDI (Universal Description, Discovery and Integration) (14%).
- Between August 2002 and February 2003, the use of Web services
 - for in-house integration by systems integrators declined by 10%
 - by system integrators for B2B solutions increased by 8%.

Gartner Dataquest survey - 2

- Web services usage has matured beyond the first stages of intra-enterprise Web services pilot projects.
- Business-process re-engineering is still the most significant challenge in implementing Web services, especially for large enterprises with more than 5,000 employees.
- Common activities for Web services projects
 - Internal integration with existing applications
 - Security, personalization (portals) and Web content management

TechMetrix: Key findings

- Adoption - Figures 1 - 2
 - There is interest in building (more) and using (less) web services but the pace varies; there has been not much change between the two surveys
- Budget - Figures 3 - 4
 - Most are “scunkworks” R&D projects
 - Of the few that have a budget, its range varies
- Technological choices - Figures 5 - 6
 - J2EE, open source and commercial

CBDI report: Organization

- Business recognize the potential of web services as a means for outsourcing IT activities
 - There are adopters of different size
 - In most all sectors
 - Throughout the world
- Figures 1 - 3

CBDI report: Adoption

- There are many completed web-services projects
- Early-adoption projects are internal and low risk, but are critical in establishing experience for subsequent projects
- In many cases it is an IT-led investment, but, in leading organizations, it is a business-driven activity
 - The two have to synchronize; currently technological impediments are
 - Security (but the projects use available security mechanisms)
 - Lack of platform and standards maturity
 - Immature industry ecosystem: neither provider nor consumer will make a move until the business case is clear
- Figures 4 - 7

CBDI report: Drivers and motivations

- How to assess web-service solutions
 - “Point tools” should be assessed with application-specific criteria on a 12month ROI basis
 - Partnerships with major tool vendors should be assessed with higher-lever criteria
 - Only use mainstream platforms and standards
 - Currently, applications are motivated mainly by platform-independence arguments, but business opportunities are increasingly becoming key
 - The arguments seem to be similar to component- and OO- based development, but it seems that organizations have a better grasp of the case with SOA and can achieve funding more easily
- Budget and plan to integrate web-service lifecycle support in 2003
 - Organizations are “making haste slowly”
- Figures 8 - 10

CBDI report: Applications

- The value is not compelling yet
 - Web services can do the same thing that other technologies can (sending SOAP messages through MQSeries servers)
- Many external projects are offering services to customers through intermediaries
- Figures 11 - 13

CBDI report: Protocol usage

- High usage of ebXML and WSIL (web-services inspection language)
- Among intermediaries, domain-specific semantic standards are important
 - The organization networks are in place
 - Various technologies have and will be adopted
- Figures 19 - 20

CBDI report: Development process

- Among end-user organizations, there is a defined lifecycle process for delivering web-service solutions
 - Requirements analysis is still the big issue (like in components- and OO-based development)
 - Thinking is moving beyond technical issues, such as reuse, to business organization (unlike with these previous technologies)
 - Testing existing services is a big issue: easy with partners, more difficult with live end-user services
- There is commitment to “service” adoption
- Figures 21 - 25

Conclusions

- Web Services is a well-conceived technology
 - Open, extendible standards promote implementation, adoption and domain-specific extensions
- The technology is quite mature
 - A variety of tools support the development and deployment of web services
- The business case seems strong, especially in domains with no pre-existing interoperability standards (EDI)